

STUDIFY Project Constitution

Core Principles

I. Architecture Boundaries

All new product work must remain inside the existing application areas: **Frontend** for user interface and client-side state, and **Backend** for APIs, services, and persistence. Do not create new root-level directories; new features, components, modules, controllers, DTOs, services, and models must be placed into the appropriate existing folders within the frontend or backend structure.

II. Stack-First Implementation

STUDIFY is currently built with a React/Vite frontend and a NestJS/TypeScript backend. The detected stack is:

- Frontend: React 19.2.6, React DOM 19.2.6, React Router 7.18.0, React Router DOM 7.18.0, Vite 8.0.12, Tailwind CSS 4.3.1, ESLint, and existing JSX/TSX component patterns.
- Backend: Node.js runtime via NestJS 11.0.1, TypeScript 5.7.3, Sequelize 6.37.8, sequelize-typescript 2.1.6, PostgreSQL drivers pg 8.21.0 and pg-hstore 2.3.4, JWT, bcryptjs, class-validator, and class-transformer. New features must reuse these technologies and prefer the libraries already declared in the workspace manifests before introducing alternatives. No third-party dependency may be installed without explicit permission from the user.

III. Naming and Organization Discipline

The existing codebase uses the following conventions:

- Variables and functions: camelCase, for example totalScore and getUserData.
- React components: PascalCase, for example LoginForm and RegisterForm.
- Files and folders: the repository primarily uses camelCase or PascalCase for source files and folders; snake_case and kebab-case are not the established convention for the main application code. New files should follow these conventions consistently and should be grouped logically within the relevant feature folders.

IV. API Contract Discipline

All new backend routes must be prefixed with /api/v1/... and must return a consistent JSON envelope: { "success": boolean, "data": object | array, "message": string } Backend changes must preserve this contract and should use DTOs and validation where appropriate.

V. Safe, Configurable, and Maintainable Code

Before editing any feature, the relevant existing files must be read and understood first. Changes must be additive and integrated smoothly so that current behavior is preserved rather than overwritten. Magic numbers and hardcoded configuration values are not allowed; secrets, URLs, credentials, and feature flags must be supplied through environment variables and .env files. For complex logic, generated code

must include concise and clear comments written in Vietnamese to support thesis explanation and review. For example: "// Đoạn logic này xử lý ..."

Additional Constraints

- Database: The current backend is configured for PostgreSQL through Sequelize. New persistence models and schemas should be created in [Backend/src/models](#) and should follow the existing Sequelize model pattern.
- State Management: The frontend currently uses a Zustand-style store pattern for authentication state in [Frontend/src/features/auth/store/useAuthStore.ts](#), alongside local component state. New frontend state logic should follow this lightweight pattern unless a different approach is explicitly approved.
- Environment Configuration: Database settings, JWT secrets, API endpoints, and other runtime-sensitive values must be supplied through .env files and must not be committed to source control.
- Documentation: When behavior changes, the relevant documentation or inline context should be updated so the next developer can understand the intent quickly.

Development Workflow

- Analyze the relevant code before editing and confirm the surrounding module structure.
- Prefer small, focused, additive changes that preserve existing functionality.
- Validate changes with the appropriate build, lint, or test commands in the relevant workspace area before marking work complete.
- If a change affects both frontend and backend, keep the interface contract and data flow consistent across both sides.

Governance

This constitution supersedes ad-hoc practices for STUDIFY. Any deviation must be documented, justified, and approved. Complexity must be explained in the change description, and implementation must remain aligned with this constitution. Amendments require updating this file and recording the revised date.

Version: 1.0.0 | **Ratified:** 2026-07-01 | **Last Amended:** 2026-07-01